

EE5203 L03 Lab Guide

Process eight sensor values with small C functions - Basri Erdogan

Mission: Copy your working L02 project and add four C files. Clamp eight sensor values, calculate their average, and write "LOW", "OK", or "HIGH". Check every result in the debugger.

Prepare before class

- ☐ Bring your working EE5203_L02_<studentnumber> project and a USB data cable.
- ☐ Review function declarations, definitions, calls, array indices, and \0.
- ☐ Download the starter files: [EE5203_L03_starter.zip](#).
- ☐ Write the answers to this entry ticket on paper before the lab:

```
uint16_t average_samples(uint16_t values[], int count);
```

1. Circle the return type, the function name, and the two parameters.
2. Why does the function need `count`? One sentence.
3. Write the valid indices when `count == 8`.

Learning objectives

By checkoff time, you can:

1. name the return type, name, and parameters in a C declaration;
2. find the declaration, definition, and call of one function in three files;
3. loop through an array without using an invalid index;
4. write a C string that fits in its array and ends with \0;
5. check a result in the debugger, then change one input and predict the new result.

Hardware and files

Item	Required value
Board	Nucleo-F401RE
Tools	VS Code (no CubeMX in this lab)
Starting point	A copy of your L02 project
Project name	EE5203_L03_<studentnumber>
Header files	sensor_math.h, status_text.h
Source files	sensor_math.c, status_text.c
Files you edit	CMakeLists.txt, main.c, sensor_math.c, status_text.c
Input	Eight fixed uint16_t samples; no ADC yet
What you show	Build output, breakpoint, VARIABLES / WATCH, MEMORY panel

Keep all edits to `main.c` inside the `USER CODE` regions.

Checkpoint A - Copy the project and add the four starter files

1. Put your EE5203_L02_<studentnumber> folder in `C:\Work\<studentnumber>`. Copy it, and rename the copy EE5203_L03_<studentnumber>. Check that the `.vscode` folder came with it.
2. In the new folder, **delete the build folder**. It still points to the L02 folder, and CMake stops with an error if you keep it.
3. Double-click EE5203_L03_starter.zip. Copy the `Core` folder you see inside and paste it into your project folder. Windows adds two files to `Core\Inc` and two files to `Core\Src`.
4. Open `CMakeLists.txt` in the project folder and make two changes.

Change the project name to the new folder name:

```
# Set the project name
set(CMAKE_PROJECT_NAME EE5203_L03_<studentnumber>)
```

Add the two new `.c` files to the build:

```
# Add sources to executable
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
  # Add user sources here
  Core/Src/sensor_math.c
  Core/Src/status_text.c
)
```

The header files need no change: `Core/Inc` is already on the include path.

5. In `main.c`, add these includes inside `USER CODE BEGIN Includes`:

```
#include <stdbool.h>
#include <stdint.h>
#include "sensor_math.h"
#include "status_text.h"
```

6. VS Code → File → Open Folder → `EE5203_L03_<studentnumber>`. Choose the **Debug** preset if asked. Build with **Ctrl+Shift+B**: zero errors.

7. Find the declaration of `clamp_u16` (file and line) and its definition (file and line).

Show: the four new files in the Explorer, the two new lines in `CMakeLists.txt`, and the build line `Linking C executable EE5203_L03_<studentnumber>.elf`.

Call the new functions from `main.c`

Inside `USER CODE BEGIN PV` in `main.c`:

```
#define SAMPLE_COUNT      8
#define STATUS_CAPACITY    8
#define SAMPLE_LIMIT      4095U
#define HIGH_THRESHOLD     2500U

uint16_t raw_samples[SAMPLE_COUNT] = {
    850U, 1250U, 4095U, 4200U,
    2050U, 90U, 3100U, 2800U
};

uint16_t bounded_samples[SAMPLE_COUNT];
char status_text[STATUS_CAPACITY];

uint16_t sample_average = 0U;
int samples_above = 0;
bool status_valid = false;
bool lab_done = false;
```

Inside `USER CODE BEGIN 2`, before the generated `while (1)`:

```
for (int i = 0; i < SAMPLE_COUNT; ++i) {
    bounded_samples[i] =
        clamp_u16(raw_samples[i], 0U, SAMPLE_LIMIT);
}

sample_average = average_samples(bounded_samples, SAMPLE_COUNT);
samples_above = count_above(bounded_samples,
                             SAMPLE_COUNT,
                             HIGH_THRESHOLD);
status_valid = make_status(status_text,
                           STATUS_CAPACITY,
                           sample_average);

lab_done = true;
```

Do not add a second infinite loop. Leave the generated `while (1)` in place.

Build again with **Ctrl+Shift+B**. Now `clamp_u16` is also **called**: find the call. If the build stops with **undefined reference** to `'clamp_u16'`, the two `.c` files are missing from `target_sources` (Checkpoint A, step 4).

Checkpoint B - Clamp and average the array safely

Complete these functions in `sensor_math.c`:

```
uint16_t clamp_u16(uint16_t value,
                  uint16_t low,
                  uint16_t high);

uint16_t average_samples(uint16_t values[],
                        int count);

int count_above(uint16_t values[],
               int count,
               uint16_t threshold);
```

Rules:

- `clamp_u16` returns `low` for values below `low`, `high` for values above `high`, and the value itself otherwise. Here the range is 0U to 4095U.
- Every array loop uses `i < count`.
- `average_samples` returns 0U when `count == 0`. It adds the values into a `uint32_t` sum, then divides once at the end.
- `count_above` counts values strictly greater than the threshold: `values[i] > threshold`, with `HIGH_THRESHOLD = 2500U`.

Before debugging, calculate the expected results on paper:

Quantity	Prediction	Observed
<code>bounded_samples[3]</code>		
<code>sample_average</code>		
<code>samples_above</code>		

Show: a breakpoint inside the loop of `average_samples`, with `i` and the sum changing as you step, and the three observed values in the table.

Checkpoint C - Write LOW, OK, or HIGH and add \0

Complete this function in `status_text.c`:

```
bool make_status(char destination[],
                 int capacity,
                 uint16_t average);
```

Rules:

- "LOW" when the average is below 1500U, "HIGH" when it is above 3000U, otherwise "OK". The capacity is eight characters, including `\0`.
- Check that the text and its `\0` fit in `capacity` **before** you write. Write the `\0` yourself.
- Return `true` only after the whole string is written. If it does not fit, write an empty string (only `\0`) when `capacity > 0` and return `false`.

Look at `status_text` in the WATCH panel and in the MEMORY panel. Fill in the character and its hexadecimal byte for indices 0 to 4:

Index	0	1	2	3	4
Character					
Byte (hex)					

Show: the string, the 0x00 byte after the last letter, and `status_valid`.

Checkpoint D - Check the final values in the debugger

Set a breakpoint on the line `lab_done = true;`. Write every prediction first. Then press **F5** and fill in the table.

Check	Where to look	Prediction	Observed
The last index used is 7	<code>i</code> at the last loop step inside <code>average_samples</code>		
The 4200 sample became 4095	<code>bounded_samples[3]</code>		
The average used all eight clamped samples	<code>sample_average</code>		
The status text ends with <code>\0</code>	<code>status_text</code> in the MEMORY panel		

Press **Step over** (F10) once: `lab_done` changes to `true`.

Checkpoint E - Individual modification

Each student gets one variant. Write your prediction first. Change only that one thing. Build, run, and compare.

Variant	Required change
A	Change the 90U sample to 5000U. Predict the clamped value and the new average.
B	Change HIGH_THRESHOLD from 2500U to 2000U. Predict samples_above.
C	Change the HIGH status boundary from 3000U to 2200U. Predict the string.
D	Call the array functions with SAMPLE_COUNT - 1. Predict which sample is left out and the new results.

Show: the new values in the debugger. Explain in two sentences why they changed.

Common problems

What you see	First check
undefined reference to 'clamp_u16'	Both .c files are listed in target_sources in CMakeLists.txt. The function name in the .c file matches the .h file.
sensor_math.h: No such file or directory	The header is in Core\Inc — not in Core\Src, and not in Core\Core\Inc.
CMake Error: The current CMakeCache.txt directory ... is different F5: program file ...EE5203_L03_<studentnumber>.elf does not exist conflicting types for 'average_samples'	You kept the copied build folder. Delete it and build again. The set(CMAKE_PROJECT_NAME ...) line matches the folder name exactly. The declaration in the .h file and the definition in the .c file use the same types.
Build is clean, but the average is wrong The status string shows extra characters	Stop inside the loop and watch i, count, and the sum. The \0 is written right after the last letter.

More setup problems: see *Common Lab Mistakes* on CATS Lessons.

Acceptance checklist

- ☐ The build is clean. Each declaration matches its definition exactly.
- ☐ Every array loop uses `i < count`. The sum is a `uint32_t`.
- ☐ The status string fits in its array, ends with `\0`, and `status_valid` is `true`.
- ☐ The prediction tables and the Checkpoint D table are filled in.
- ☐ Your individual modification works and you can explain it.
- ☐ All `main.c` edits are inside the `USER_CODE` regions.

Individual oral check

Be ready for one question and one live change:

- What is the difference between declaring and defining a function?
- Why can `average_samples` not find the length of the array by itself?
- Why is the sum a `uint32_t`? Where is the `\0` byte in your status array?
- The build has 0 errors. Does that mean the average is correct? How do you check it?
- You see `undefined reference to 'count_above'`. Which file do you check first?

Submit

Submit the four module files, `CMakeLists.txt`, the `main.c` user-code sections, one debugger screenshot of the final arrays and results, the completed tables, and a three-sentence explanation of your modification.